
大数据与人工智能
自动化学院

Chapter03.08 正则化与标准化

3.8-1 导言

在之前的章节中，我们已经搭建了完整的训练流程。本章我们将学习两个非常重要的“高级技巧”，它们能让我们的模型训练得更好、更稳定。

1. 正则化 (Regularization): 这是一系列防止模型“死记硬背”（即**过拟合**）的方法。它通过给模型增加一些“约束”，鼓励模型学习到更通用、更简单的规律。
    - 我们将学习两种主流的正则化方法：
        - **权重衰减 (Weight Decay / L2正则化)**
        - **随机失活 (Dropout)**

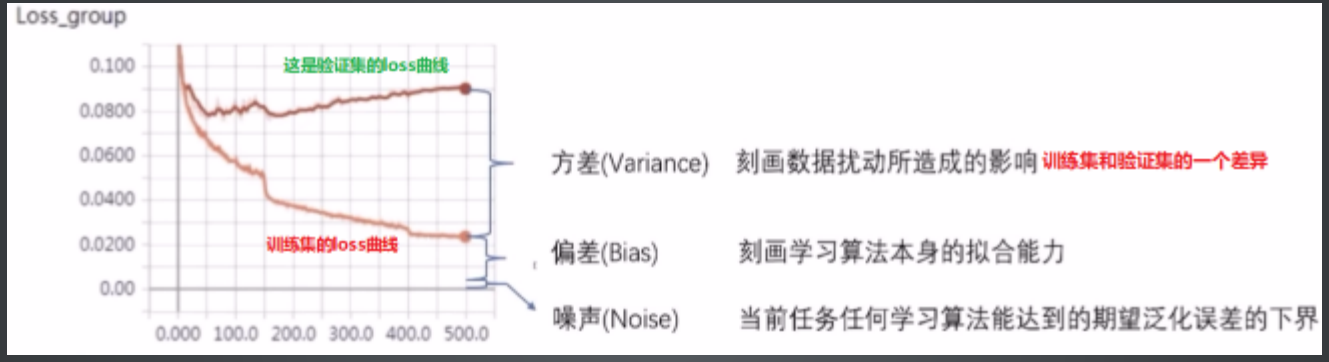
2. 标准化 (Normalization): 这是一种统一网络内部数据“度量衡”的技术。它能让数据在网络层之间流动时保持稳定的分布，从而极大地加速训练过程并提升稳定性。
    - 我们将重点学习：
        - **批标准化 (Batch Normalization)** 及其变体。

3.8-2. 正则化weight_decay（L2正则）

正则化：就是一个减少模型的**方差**的策略。

- 模型的总误差可以分解为三部分：偏差、方差和噪声。
  - **偏差 (Bias):** 描述了模型的**基础学习能力**。高偏差就像一个“学渣”，连训练集上的简单规律都学不好，这叫**欠拟合**。
  - **方差 (Variance):** 描述了模型对训练数据**有多敏感**。高方差就像一个“书呆子”，把练习册上的所有题目都背得滚瓜烂熟，但一到考场遇到新题型就傻眼了。这种在训练集上表现完美，但在新数据（验证集）上表现很差的现象，就叫**过拟合**。

看下面这个图：



- 通常所说的过拟合现象，也就是指的高方差，就是模型在训练集上训练的超级好，几乎全部都能拟合。
- 但是进行实际应用时候，效果却很差

- **我们的目标：** 找到一个低偏差、低方差的平衡点。而**正则化**就是用来降低方差，解决过拟合问题的主要策略。
- 正则化的思想就是在我们的目标函数中增加正则项，

  即： $$\text{Obj} = \text{Cost} + \text{Regularization}$$

还记得我们目标函数呢？ 其中这里的代价函数Cost是

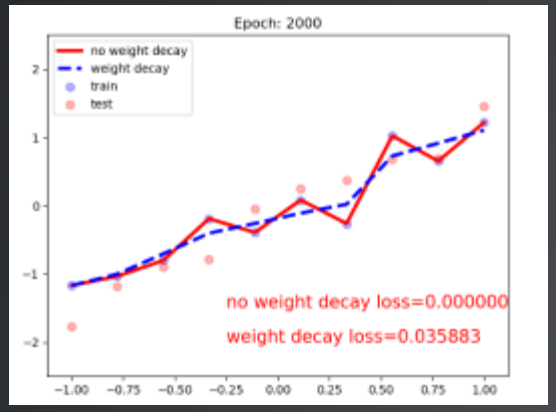
$$\text{cost} = \frac{1}{N}\sum_{i}^{N} f(\hat{y_i}, y_i)$$

我们这里的正则项部分就是有L1正则项和L2正则项两种。 我们先来看看两者的表达式：

- L1 Regularization: $\sum_i^N |w_i|$
- L2 Regularization: $\sum_i^N w_i^2$

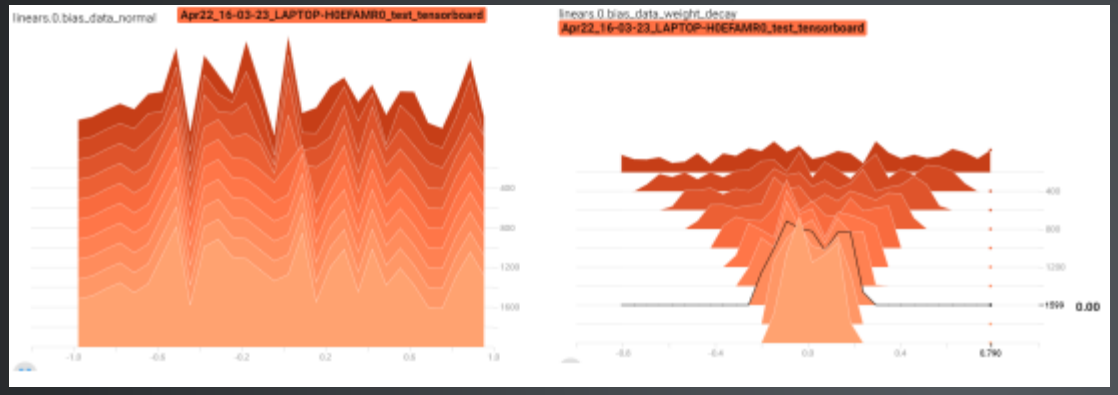
- 这里wi表示在模型中第i个输入的权重（贡献）

我们可以看一下正则化模型和不带正则化模型的效果：



- 结果是对于test数据集（红点）的拟合：
- 红线：不带正则化的优化。基本都没有拟合到红点
- 蓝线：带正则化的优化。拟合到的红点稍多（比红线稍好）
- 可以清楚的发现， 不带正则化的红色线发生了过拟合现象。

下面通过Tensorboard观察一下梯度参数的一个分布情况，这里面可以明显的看出衰减表示的意思：



- 左边的是不带正则化的模型参数的分布情况，我们可以看到从迭代开始到结束整个权值的分布都没有什么变化
- 右边是加入了weight decay的分布， 可以看到衰减的趋势， 这说明L2正则起作用了，使得在迭代过程中权值在不断的缩减，以至于模型不会过于复杂产生过拟合。

总结:

- 想获得一个**稀疏模型**或进行**特征选择**，用L1。
- 想获得一个**更准确、泛化能力更强**的模型，且所有特征都有用，用L2。L2在深度学习中更常用。

随堂作业

L1正则化与L2正则化的区别主要体现在什么方面?

- A. 计算复杂度
- B. 对异常值的敏感性
- C. 权重稀疏性
- D. 收敛速度

参考答案：C

选项分析：

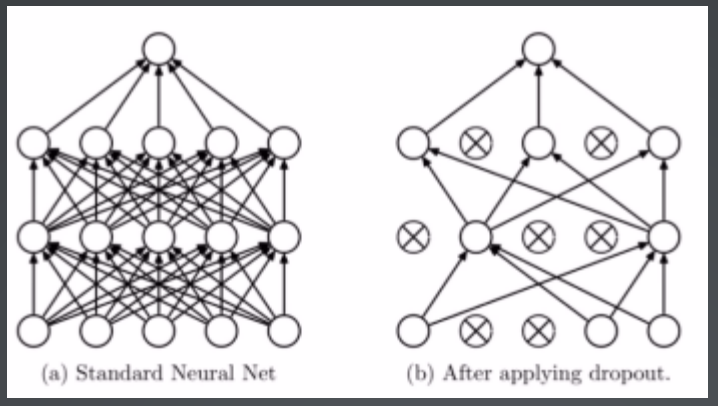
- A: L1正则化计算复杂度较高，但这不是主要区别。
- B: L2正则化对异常值更为敏感，而L1较具抗性
- C: L1正则化倾向于产生稀疏解，即部分权重为零，而L2通常不会
- D: 收敛速度与具体优化算法有关，不是L1和L2的直接区别。

- Dropout 是一种非常有效的正则化技巧。在**训练过程中，它会以一定的概率 p 随机地“关闭”或“失活”一部分神经元**。

3.8-3.1 Dropout概念

Dropout叫做随机失活: 就是给出一个概率(随机)，让某个神经元的权重为0(失活)。

下面给出个图直观感受一下:



- Dropout就是每一层，让某些神经元不起作用

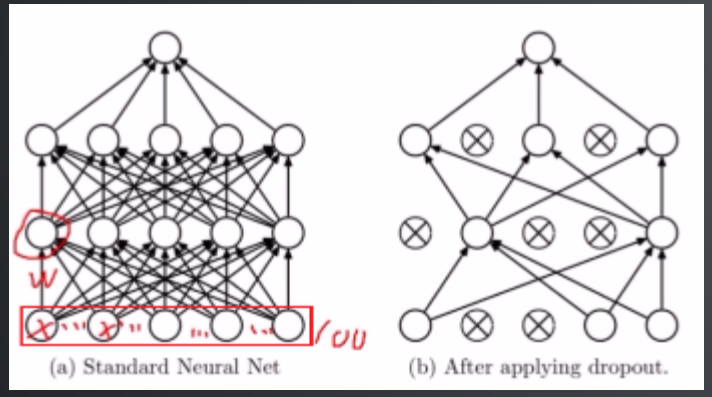
- 这样就就相当于把网络进行简化了(左边和右边可以对比)

- 我们有时候之所以会出现**过拟合现象**
  - 因为**网络太复杂了，参数太多了**
  - 并且我们后面层的网络也可能**太过于依赖前层的某个神经元**
  - **比喻：** 就像一个团队在合作学习，每次练习时，都随机让几个成员“休息”。这样一来，剩下的成员就不能过分依赖某个“学霸”，每个人都必须独立地学习，从而提升整个团队的综合实力。
  - 这使得网络变得不那么依赖于某些特定的神经元组合，迫使它学习到更加鲁棒和泛化的特征，从而有效缓解过拟合。

- **训练与测试的差异:**
  - **仅在训练时使用Dropout！** 在测试（或实际应用）时，我们需要模型拿出全部实力，所以会**激活所有神经元**。
  - 这就带来一个问题：训练时因为部分神经元关闭，某一层输出的总能量会变小；而测试时所有神经元都工作，总能量会变大，造成了**尺度不匹配**。

- **所以测试的时候，所有权重都乘以1-drop_prob， 以保证训练和测试时尺度变化一致**，这里drop_prob是随机失活概率。

这个权重都乘以1-drop_prob应该怎么理解呢？ 依然拿上面那个图来说：



看上面这个图，假设我们的输入是100个特征， 那么第一层的第一个神经元的表达式应该是这样， 这里先假设不失活：

$$Z_1^1 = \sum_{i=1}^{100} w_i x_i$$

- 假设我们这里的wi*xi=1， 那么第一层第1个神经元 Z11 =100， 注意这是不失活的情况，

- 那么如果失活呢？ 假设失活率drop_prob=0.3， 也就是我们的输入大约有30%是不起作用的，也就是会有30个不起作用，因为失活率%30指的是每个神经元的失活率。
- 那么我们的 Z11相当于 $$Z_{1\ \text{train}}^{1} = \sum_{i=1}^{70} w_i x_i = 70$$

我们发现，如果使用Dropout之后，我们的Z11成了70， 比起不失活来少了30， 这就是一个尺度的变化，

- 所以如果训练的时候用Dropout， 我们每个神经元取值的尺度是会缩小的，比如这里的70，
- 而测试的时候我们用的是全部的神经元，尺度会变成100，这就导致模型在数值上有了差异

因此，在测试的时候，需要所有的权重乘以1-drop_prob这一项

这时候我们在测试的时候就相当于：

$$Z_{1\ test}^{1} = \sum_{i=1}^{100} (0.7 \times w_i) x_i = 0.7 \times 100 = 70$$

这样采用Dropout的训练集和不采用Dropout的测试集的尺度就变成一致了

后面介绍，如何实现Dropout呢？

提问：在使用Dropout时，为什么测试阶段权重需要调整?

- A. 为了增加模型复杂度
- B. 为了保持训练和测试阶段的一致性
- C. 为了减少计算量
- D. 为了提高模型精度

参考答案：B

选项分析：

- A: Dropout用于简化模型，而非增加复杂度。
- B: 在测试阶段，所有神经元都被激活，因此需要调整权重以保持与训练阶段一致的数据尺度
- C: 调整权重不是为了减少计算量。
- D: 精度提高是调整后的结果，但不是直接原因。

3.6-3.2 nn.Dropout

Pytorch提供了Dropout层， nn.Dropout

```
torch.nn.Dropout(p=0.5, inplace=False)
```

这里的p就是被舍弃概率，也就是失活概率。

下面就用上面正则化L2的代码实例看看不用L2，而是加上Dropout的效果:

```
# ============================== step 1/5 数据
def gen_data(num_data=10, x_range=(-1, 1)):

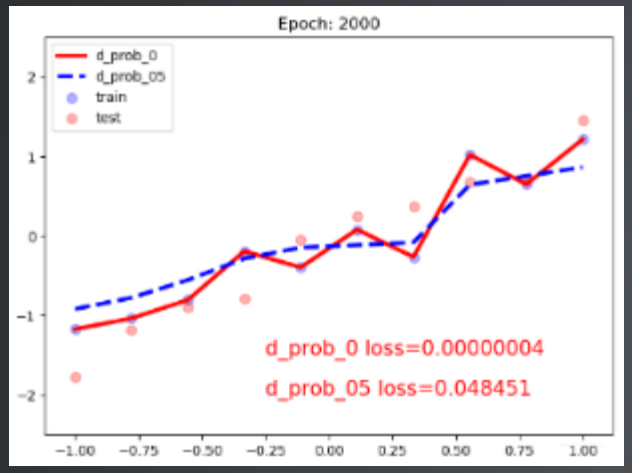
    w = 1.5
    train_x = torch.linspace(*x_range, num_d
    train_y = w*train_x + torch.normal(0, 0.
    test_x = torch.linspace(*x_range, num_da
    test_y = w*test_x + torch.normal(0, 0.3,
```

```
return train_x, train_y, test_x, test_y
```

所以上面的代码里面要注意两点，

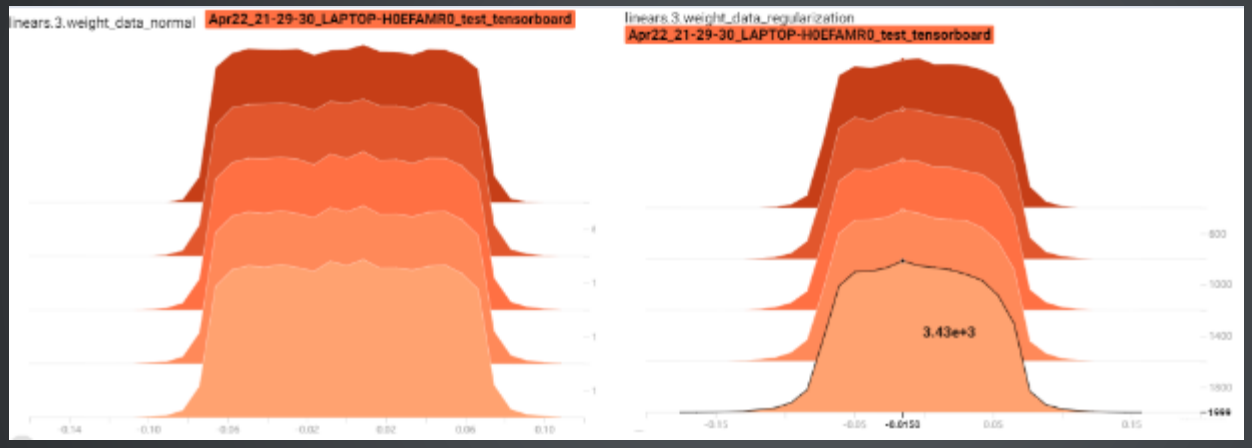
- 第一点：Dropout加的时候注意放置的位置
- 第二点：由于Dropout操作，模型训练和测试是不一样的，训练的时候采用Dropout而测试的时候不用Dropout，
  - 在迭代的时候，就得告诉网络目前是什么状态，
  - 如果要测试，就得先用`.eval()`函数告诉网络，现在是测试，不是训练
  - 训练的时候就用`.train()`函数告诉网络，现在是训练网络。

- 我们下面看一下Dropout正则化后的效果：



- 可以清楚的发现， 不带Dropout（dropout概率为0）的红色线发生了过拟合现
- 但是带有Dropout的效果和L2正则差不多

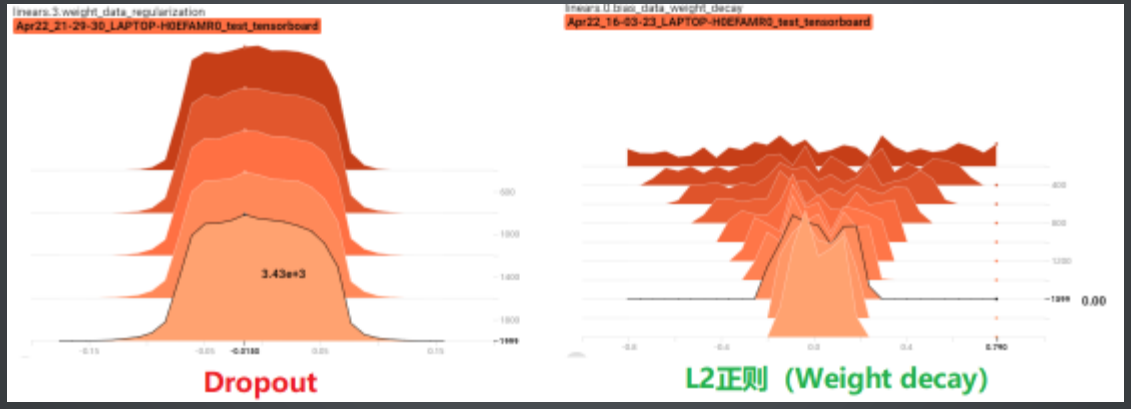
- 下面通过Tensorboard观察一下梯度参数的一个分布情况：



- 从上面这两个图，也可以看出Dropout有利于收缩权重的分布。 类似于L2的功能，

但是这里可看不出衰减

我们对比一下Dropout和L2：



上面就是如何在Pytorch中如何使用Dropout，但是在实现上，还有一个小细节得说一下：

- PyTorch的做法更聪明：它在**训练时**，就把那些“存活”下来的神经元的输出值**放大** 1/(1-p) 倍。这样一来，无论训练还是测试，该层输出的期望总能量都保持一致，测试时就无需做任何额外操作了。
- 这样就不用再测试的时候权重乘以1-p， 也没有改变原来数据的尺度。
- 比如：上面公式中，p = 0.3，每个激活的权重项修改为 $\frac{1}{1-0.3} \cdot w_i \cdot x_i$，结果如下：
  $Z_{\text{train}} = \sum_{i=1}^{70} \left(\frac{1}{0.7} w_i x_i\right) = \sum_{i=1}^{70} (1.42857 \cdot w_i \cdot x_i) = 10$
  $Z_{\text{test}} = \sum_{i=1}^{100} w_i x_i = 100$
- 上面训练集和测试集的尺度依然是相等，并且还和原来数据的权重之和相等了。

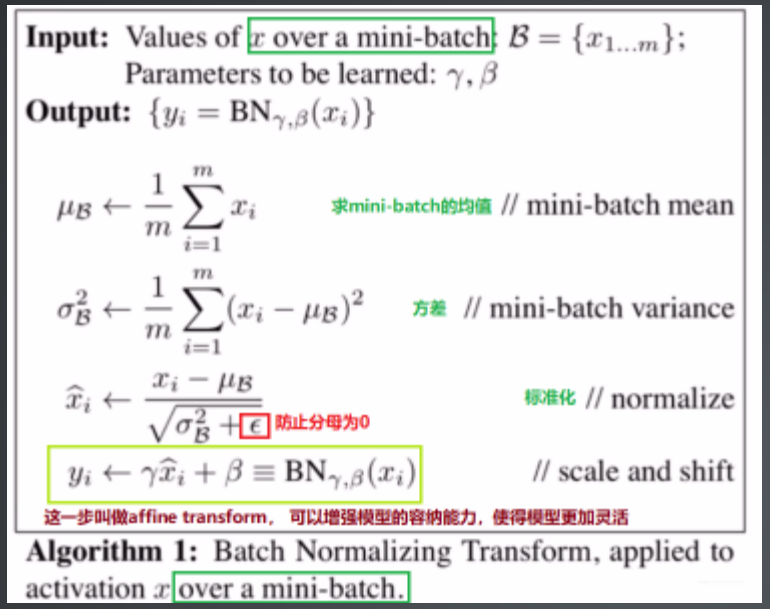
3.8-4. 标准化之BatchNormalization

3.8-4.1 BatchNormalization是什么以及为啥用？

BatchNormalization就是批标准化， 批指的是mini-batch, 标准化也就是均值为0方差为1

- 在复杂的深度网络中，每一层的输出分布会因为上一层参数的更新而不断变化，这种现象称为**内部协变量偏移(Internal Covariate Shift)**。这就像让每一层网络都在追逐一个“移动靶”，增加了训练难度。
- **Batch Normalization (BN)** 的作用就是在数据进入下一层网络之前，强行将其“拉回”到一个**标准的、稳定的分布**上（通常是均值为0，方差为1）

下面就是BatchNormlization的算法流程：



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$$ 求mini-batch的均值 // mini-batch mean

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$ 方差 // mini-batch variance

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$ 防止分母为0 标准化 // normalize

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i)$$ // scale and shift

这一步叫做affine transform，可以增强模型的容纳能力，使得模型更加灵活

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

最后这句称为affine transform, 可以增强模型的容纳能力，使得模型自己更加灵活，让模型自己去判断是否需要去改变数据的一个分布（这里的 γ和 β是可学习参数， 类似于神经网络的权值 w和 b）

- 看似这个东西比较简单，但是威力却是很强
- 有下面几个优点（Ref:《BatchNormalization：Accelerating Deep Network Train by Reducing Internal Covariate Shift》，经典论文）：
  - 可以**用**更大学习率，加速模型收敛
  - 可以**不用**精心设计权值初始化
  - 可以**不用**Dropout或者较小的Dropout
  - 可以**不用**L2或者较小的weight decay
  - 可以**不用**局部响应标准化（AlexNet中用到过）

- 工作流程:
  - 对一批（batch）数据，计算其均值和方差。
  - 使用该批次的均值和方差，对每个数据点进行标准化。
  - 通过两个可学习的参数 γ (gamma) 和 β (beta)，对标准化后的数据进行缩放和平移。这一步让网络有能力“反悔”，如果它认为原始分布更好，可以通过学习让 γ 和 β 将数据恢复原样。

BatchNormlization（BN）既然这么简单，为啥能起那么大的作用呢?

下面就通过代码看一下，加了BN之后，为啥不用精心的设置权值初始化了， 依然是权值初始化那里的那份代码:

```
class MLP(nn.Module):
    def __init__(self, neural_num, layers=10
        super(MLP, self).__init__()
        self.linears = nn.ModuleList([nn.Lin
        self.bns = nn.ModuleList([nn.BatchNo
        self.neural_num = neural_num

    def forward(self, x):

        for (i, linear), bn in zip(enumerate
            x = linear(x)
```

首先我们先不用权值初始化，不用BN， 看看网络出现的问题：

ation.py | bn_and_initialize.py | bn_in_123_dim.py | bn_application.py | optimizer.py

```
15        self.neural_num = neural_num
16
17    def forward(self, x):
18
19        for (i, linear), bn in zip(enumerate(self.linears), self.bns):
20            x = linear(x)
21            # x = bn(x)
22            x = torch.relu(x)
23
24            if torch.isnan(x.std()):
25                print("output is nan in {} layers".format(i))
26                break
27
28            print("layers:{}, std:{}".format(i, x.std().item()))
29
30        return x
```

```
layers:87, std:2.2138910767543786e-35
layers:88, std:8.807052062679093e-36
layers:89, std:3.432233645943062e-36
layers:90, std:1.3510364667986602e-36
layers:91, std:5.099643038273595e-37
layers:92, std:1.866536280713285e-37
layers:93, std:7.515118822623136e-38
layers:94, std:2.6169069455062525e-38
layers:95, std:1.1516198683661999e-38
layers:96, std:4.344905254842529e-39
layers:97, std:1.5943497485609898e-39
layers:98, std:5.721221370145363e-40
layers:99, std:2.4877251637158477e-40
tensor([[0.0000e+00, 2.1160e-41, 0.0000e+00,  ..., 0.0000e+00, 0.0000e+00,
```

可以发现，到了后面网络的输出非常小非常小了，这样很容易导致梯度消失

那么我们加上权值初始化，由于网络里面用到了relu， 所以这里使用Kaiming初始化方法， 看下效果：

zation.py | bn_and_initialize.py | bn_in_123_dim.py | bn_application.py | optimizer.py

```
15        self.neural_num = neural_num
16
17    def forward(self, x):
18
19        for (i, linear), bn in zip(enumerate(self.linears), self.bns):
20            x = linear(x)
21            x = bn(x)
22            x = torch.relu(x)
23
24            if torch.isnan(x.std()):
25                print("output is nan in {} layers".format(i))
26                break
27
28            print("layers:{}, std:{}".format(i, x.std().item()))
29
30        return x
```

```
layers:86, std:0.703682005405426
layers:87, std:0.7321067452430725
layers:88, std:0.6924356818199158
layers:89, std:0.6652532815933228
layers:90, std:0.6728308796882629
layers:91, std:0.6606621742248535
layers:92, std:0.6094604730606079
layers:93, std:0.6019102334976196
layers:94, std:0.595421552658081
layers:95, std:0.6624555587768555
layers:96, std:0.6377885341644287
layers:97, std:0.6079285740852356
layers:98, std:0.6579315066337585
layers:99, std:0.6668476462364197
```

可以发现数据的尺度
就有一个比较好的分布了

这里是我们精心设计了权值初始化的一个方法

- 如果激活函数是relu, 得用Kaiming初始化
- 如果激活函数是tanh，还得用Xavier， 还是挺复杂的，

- 那么假设不用权值初始化，而是在网络层的激活函数前加上BN

```
        return x

    def initialize(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):

                # method 1
                nn.init.normal_(m.weight.data, std=1)    # normal: mean=0, std=1

                # method 2 kaiming
                # nn.init.kaiming_normal_(m.weight.data)
```

```
layers:85, std:0.5865289568901062
layers:86, std:0.5752704739570618
layers:87, std:0.5757707357406616
layers:88, std:0.5795237421989441
layers:89, std:0.5833550095558167
layers:90, std:0.5762044787406921
layers:91, std:0.5773496627807617
layers:92, std:0.5739866495132446
layers:93, std:0.5821447968482971
layers:94, std:0.5786625742912292
layers:95, std:0.5817692875862122
layers:96, std:0.5800743103027344
layers:97, std:0.5726175308227539
layers:98, std:0.5763944983482361
layers:99, std:0.5823305249214172
```

可以发现数据的尺度也是在一个合理的范围之内

- 可以发现，强大的BN依然可以保证数据的尺度，并且好处就是我们不用再考虑用什么样的方式进行权值的初始化。

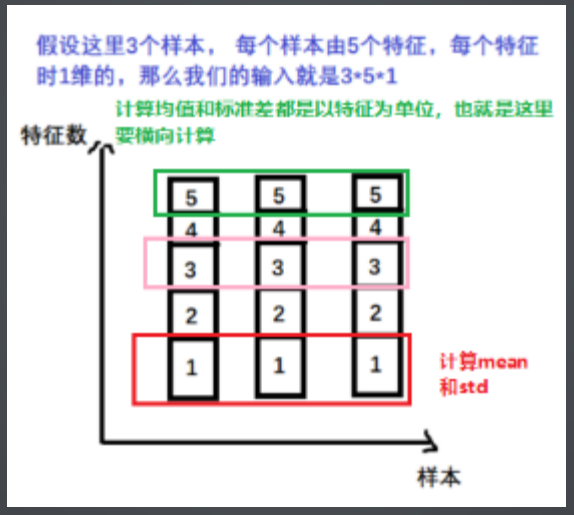
3.8-4.2 Pytorch中的BatchNormalization

Pytorch中提供了三种BatchNorm方法：

- nn.BatchNorm1d
- nn.BatchNorm2d
- nn.BatchNorm3d

下面看看nn.BatchNorm1d~3d这三个方法的区别?

- nn.BatchNorm1d -> input = B * 特征数 * 1d特征



- B 代表一个批次Batch里面，样本的数目

下面直接用代码看看上面这个一维的这个BN方法要怎么用：

```
batch_size = 3     # 3个样本
num_features = 5     # 5个特征
momentum = 0.3     # 这个计算后面均值，方差的

features_shape = (1)    # 我们特征的维度是1

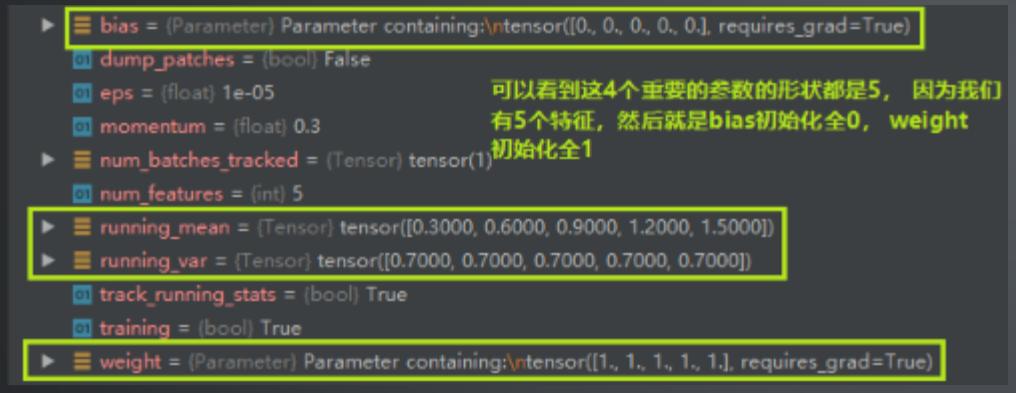
feature_map = torch.ones(features_shape)
feature_maps = torch.stack([feature_map*
feature_maps_bs = torch.stack([feature_m
print("input data:\n{} shape is {}".form
```

下面看一下结果：

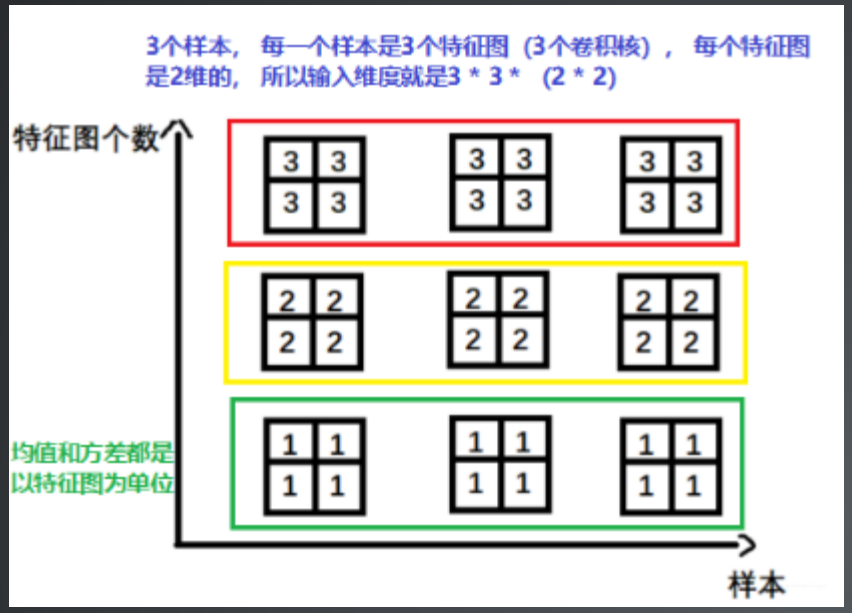


- **当前mini-bath所得到的用于对数据进行Normalize的这个均值，不是当前mini-batch得到的均值，而是会考虑前面mini-batch的数据信息，加权平均的一个均值和方差。**

下面看看BN里面的四个重要的参数：均值，方差， gamma和beta它们的shape：



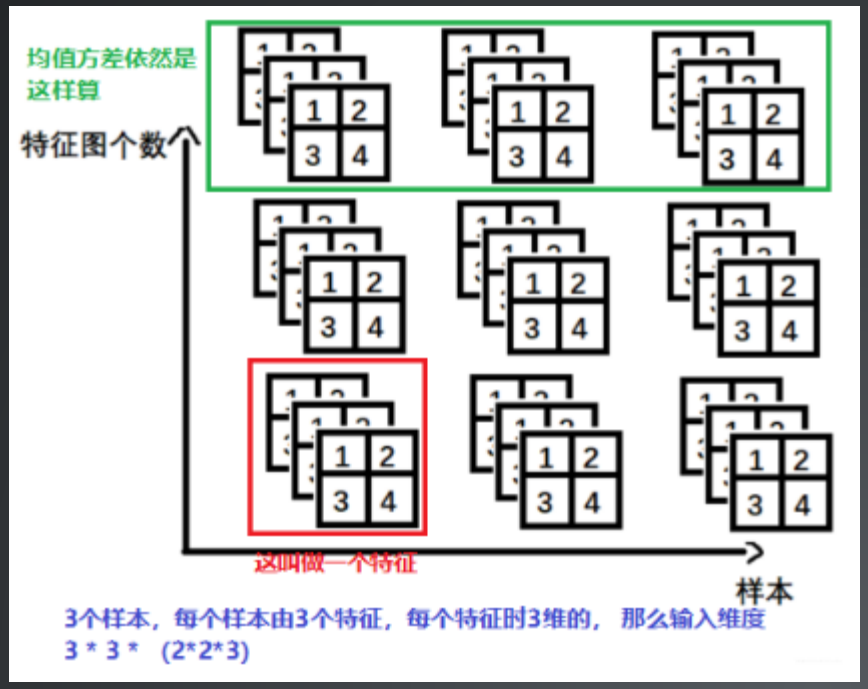
- nn.BatchNorm2d -> input = B * 特征数 * 2d特征



卷积图像中经常是这种2d的。

- 横轴表示多个样本
- 纵轴表示每个样本的特征：比如RGB
- 每个方块单元里面都是一个图片：比如某个样本的R通道的图像

- nn.BatchNorm3d -> input = B 特征数 3d特征



这个在时空序列(比如视频)中会用到

- 横轴表示多个样本
- 纵轴表示每个样本的特征：比如RGB
- 每个方块单元里面都是一个视频：比如某个视频的R通道的时间序列图像

这就是BatchNormalization的内容了，下面再学习一些其他的Normalization的一些知识。

3.8-4.3 其他的Normalization方法

我们常见的Normalization方法其实有四种

- Batch Normalization(BN)
- Layer Normalization(LN)
- Instance Normalization(IN)
- Group Normalization(GN)

这四种方式既然都是Normalization，那么有什么相同点和异同点呢?

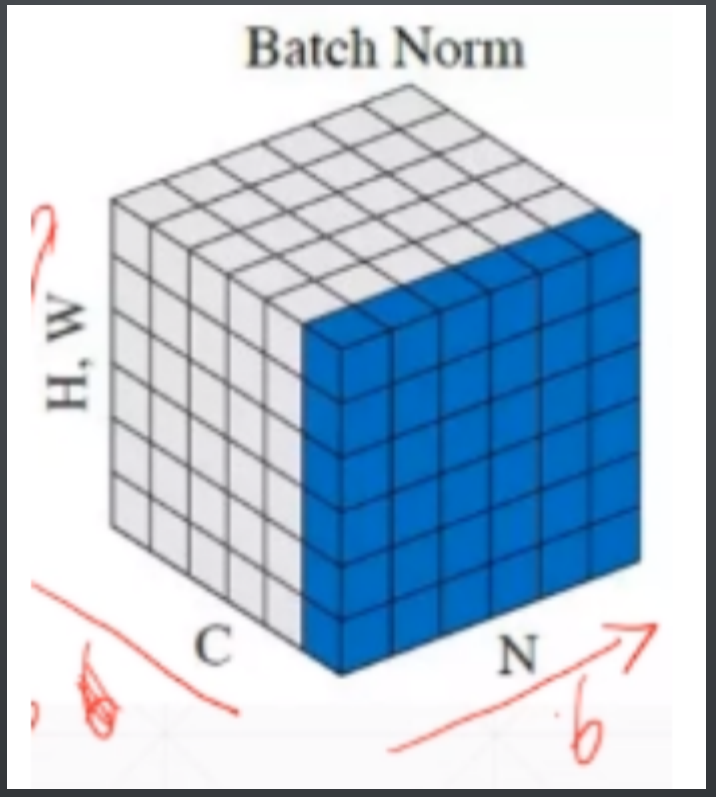
- 相同点就是公式上：

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$
$$y_i \leftarrow \gamma \widehat{x}_i + \beta) \equiv \mathbf{N}_{\gamma,\beta}\left(x_i\right)$$

- 不同点，就是求取μB和 σB的方式不同。
- BatchNormalization已经学习了，是在一个batch上去计算均值和方差
- Layer Normalization是以层为单位去计算均值和方差
- Instance Normalization主要在图像生成方法中使用的一个方法
- Group Normalization是按组为单位计算均值和方差

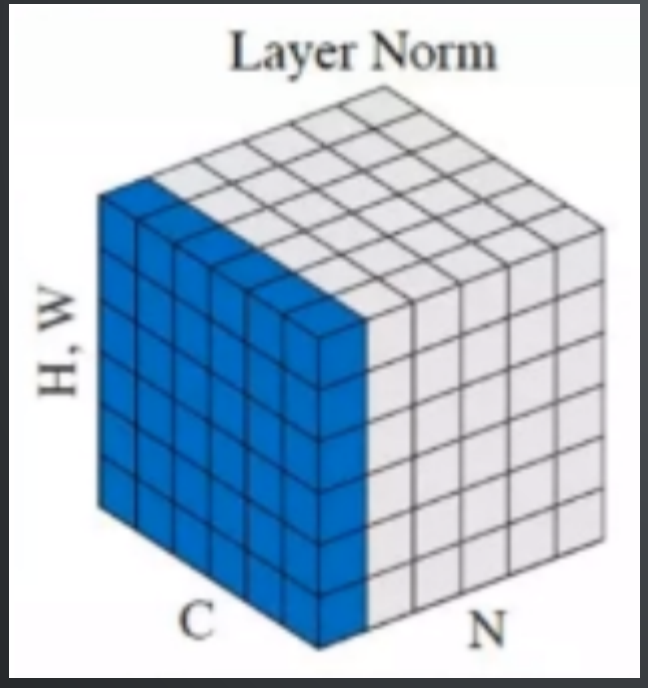
批标准化 (Batch Norm, BN):

- **计算范围:** 对每个**通道(C)，计算其在所有样本(N)** 和所有空间位置(H, W)上的均值和方差 。
- **特点:** 效果好，但非常依赖批次大小（batch size）。批次太小会导致统计量不准。



层标准化 (Layer Norm, LN):

- **计算范围:** 对每个**样本(N)，计算其在所有通道(C)** 和所有空间位置(H, W)上的均值和方差 。
- **特点:** 完全不依赖批次大小。在RNN和Transformer（如LLM）中表现优异 。



下面我们看看Pytorch中的LN：

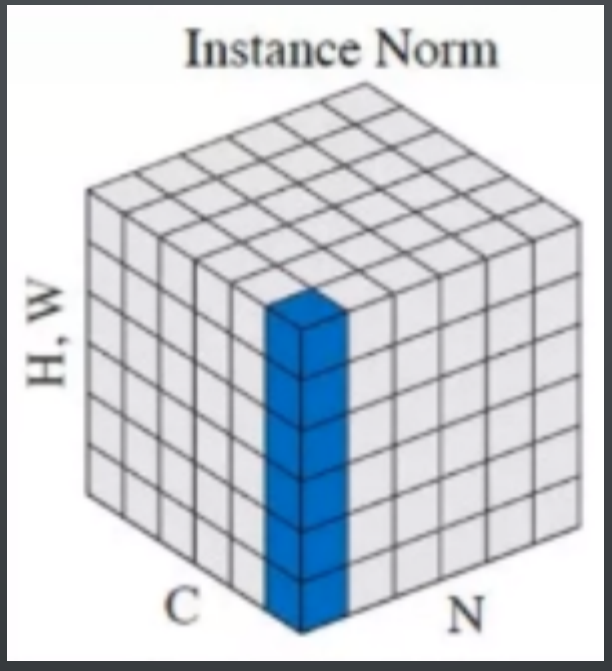
```
nn.LayerNorm(normalized_shape, eps=1e-05, el
```

- normalized_shape表示该层特征形状(C方向的维数)，这个依然是最重要的。
- eps表示分母修正项，
- elementwise_affine表示是否需要affine transform。

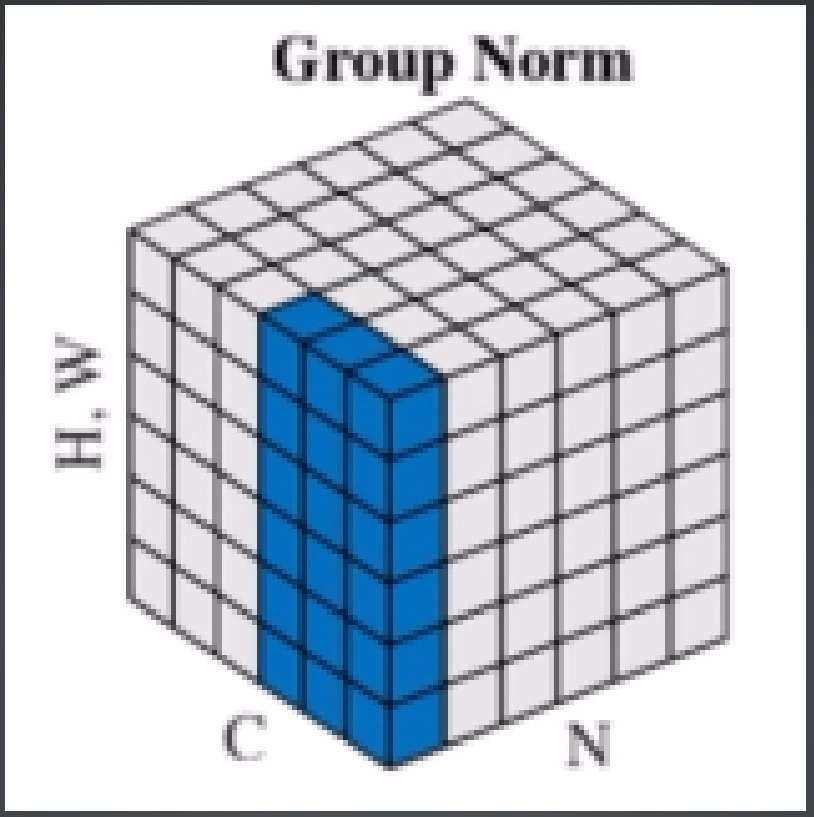
- LayerNorm 常用在RNN当中，在CNN和FCN中不如BN的效果好。
- 在Large Language Model(LLM)中应用较好

3. **实例标准化 (Instance Norm, IN):**

- **计算范围:** 对每个样本的**每个通道**独立计算其在空间位置(H, W)上的均值和方差 。
- **特点:** 常用在风格迁移等图像生成任务中，因为它保留了单个样本单个通道的独立风格信息。



4. 组标准化 (Group Norm, GN):



- **计算范围:** 是LN和IN的折中。它先将**通道(C)** 分成若干个**组(Group)**，然后对每个样本的每个组独立地计算均值和方差。
- **特点:** 同样不依赖批次大小，是BN在小批次场景下的一个优秀替代方案。

Pytorch提供的GroupNorm：

```
nn.GroupNorm(num_groups, num_channels, eps=1
```

- num_groups表示分组数
- num_channels表示特征数
- eps表示分母修正项，
- affine是否需要affine transform。
- 这里前两个参数是必须的，要不然机器也不知道你要咋分组，每组多少个等。
- 提供了组数和通道数，两者的商就代表一组是多少个特征， 这样从一组中求平均和方差。
- 还要注意分组的时候要能整除。

- BN， LN， IN和GN都是为了克服 **Internal Covariate Shift(ICS)** 提出来的，它们的计算公式差不多，只不过计算均值和方差的时候采用的方式不同，

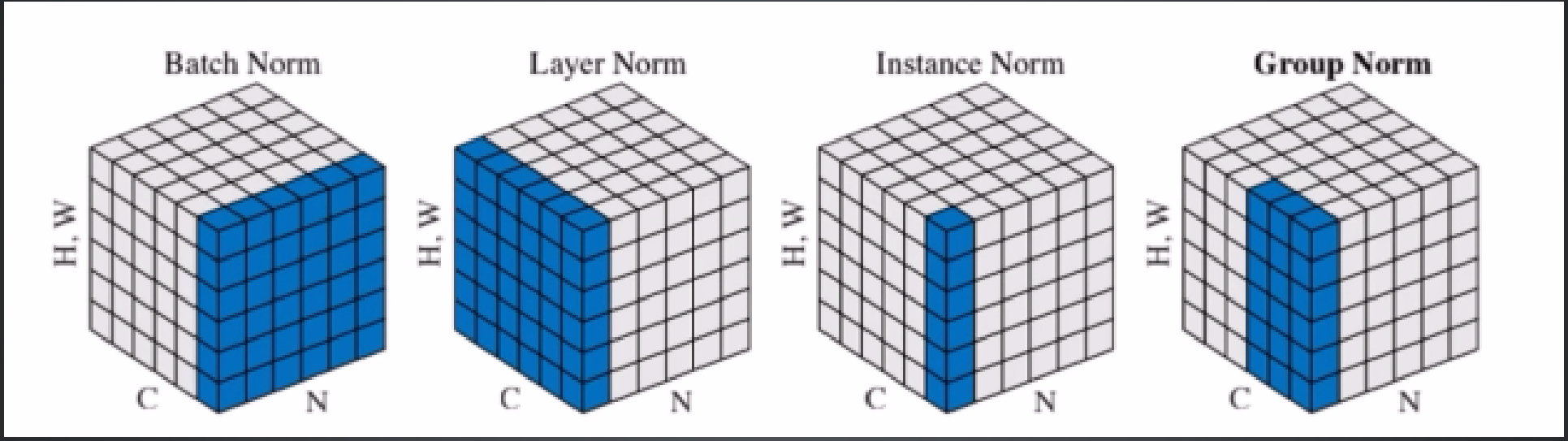
课堂作业

在RNN或Transformer等处理变长序列的模型中，由于批次内各个样本长度不同，通常使用_____标准化而不是批标准化。

答案: 层 / Layer

- **解析**: 层标准化(LN)对单个样本的所有特征进行标准化，不依赖于批次中的其他样本，因此天然适合处理变长序列。

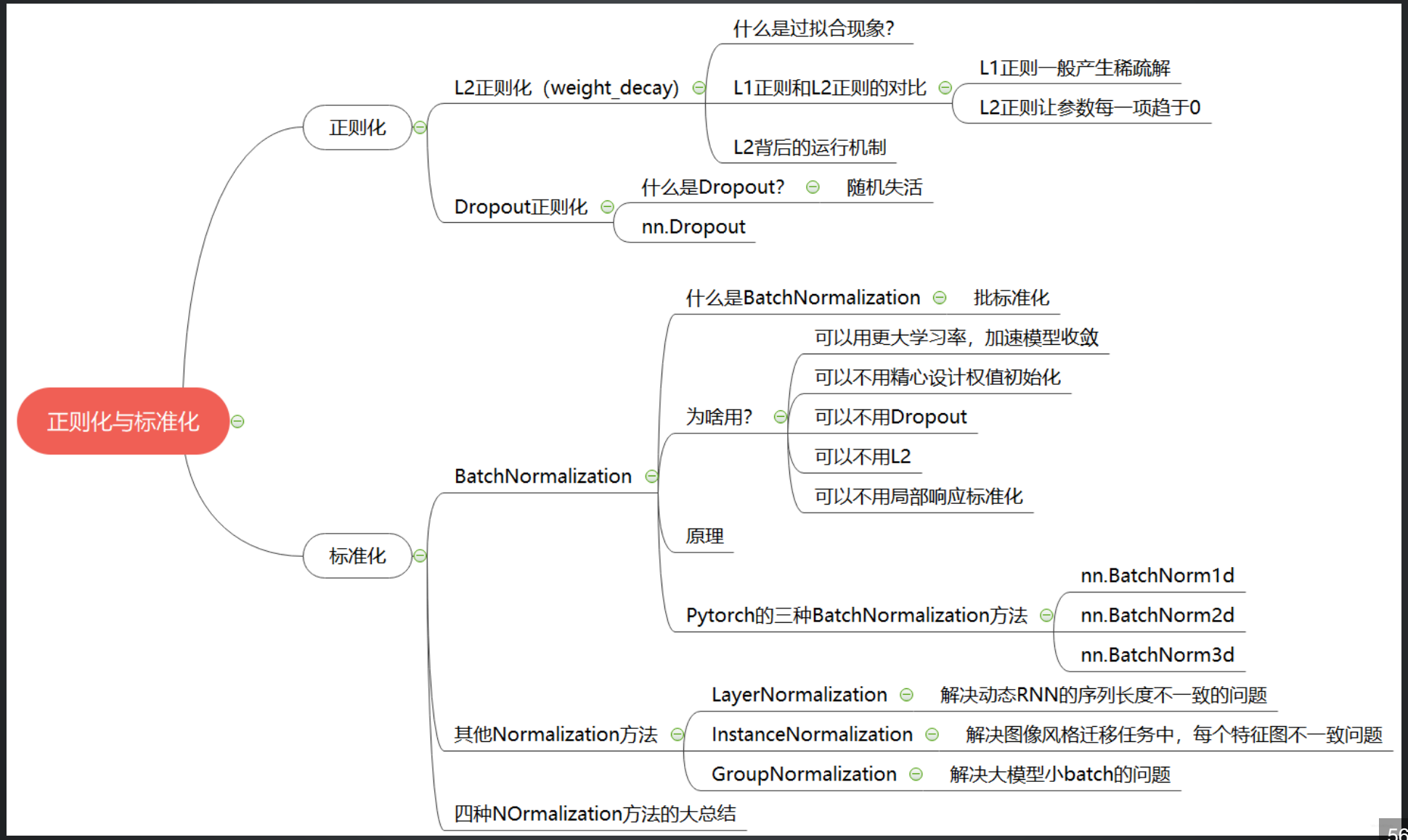
下面来整理一下：



关于这四种方法的详细介绍，可以参考这篇文章
https://blog.csdn.net/liuxiao214/article/details/81037416

3.8-5. 总结

下面依然是一张思维导图把知识拎起来：



快速梳理一遍

- 首先是正则化
  - 正则化主要是缓解模型的过拟合问题
  - 从L2正则化开始，L2正则化也叫权重衰减，我们学习了L2正则的原理，
  - L1正则和L2正则的区别，然后学习了L2正则在Pytorch中的使用。
  - 然后又学习了Dropout正则化，依然是原理和使用，并且对比了一下L2正则和Dropout正则的效果。

- 标准化主要是解决网络层输出的数据尺度变化不一致的问题，
  - 首先学习了Batch Normalization，这个非常重要，有很多的优点， 学习了它的原理和具体的使用方法，
  - 然后又介绍了其他三种标准化方法， LayerNorm Normalization、Instance Normalization和Group Normalization， 分别看了一下是如何计算的并且在Pytorch中如何使用的。
  - 最后对比了一下这四种方法。

最后再安排一节，综合上面五个步骤搭建一个Pytorch模型并且进行有效的训练

- 模型搭建完了之后要保存下来，以备后面的使用，并且在大型任务中我们不可能从头自己搭建/训练模型，往往需要模型的迁移(微调)
- 为了提高训练效率，我们往往需要使用GPU